

ROYAL NAVY · SITE EDUCATIVO

Documentação Técnica Completa

Auditoria, correções de desempenho, nova identidade visual
e dossiê de construção do site



Repositório

github.com/AlanSouzaDev7/Naval_Design-WEB-V.1

Versão

Versão 1.1 · 20 de setembro de 2026 · site publicado no GitHub Pages

Sumário

1	Resumo executivo	resultados, métricas e o que foi entregue
2	Dossiê técnico: como o site foi construído	arquitetura, arquivos e o papel de cada linguagem
3	O motor 3D em detalhe	Three.js, WebGL, ciclo de vida das cenas
4	O hero animado e as demais animações	Canvas 2D, reveal, contadores, modal
5	Diagnóstico de desempenho	o que deixava o site lento e como foi resolvido
6	Erros e bugs corrigidos	catálogo completo com causa e solução
7	Nova paleta: dia ensolarado no mar	cores, aplicação e acessibilidade
8	Como foi verificado	metodologia, medições e limitações
9	Pendências e recomendações	o que ficou de fora e próximos passos
10	Operação: executar, versionar e publicar	comandos, variáveis e GitHub
11	Publicação, domínio e segurança	GitHub Pages, royalnavy.com.br, CSP e checklist
12	Glossário	termos técnicos usados neste documento

Como ler este documento. As seções 1 a 4 explicam *o que é o site e como ele funciona*; as seções 5 e 6 registram *o que estava errado e o que foi feito*; a seção 7 documenta a nova identidade visual; as seções 8 a 11 cobrem verificação, pendências, operação e a publicação na internet (incluindo o domínio próprio). Todos os números citados foram medidos ou extraídos do código-fonte durante o trabalho — nada é estimativa sem indicação.

1. Resumo executivo

O projeto **Royal Navy** é um site educativo sobre a Marinha Real Britânica, com linha do tempo, navios lendários em **3D interativo**, batalhas, frota atual e galeria. Era servido por Flask (Python) e escrito em HTML, CSS e JavaScript puros, com o Three.js para os modelos 3D.

Foram solicitadas cinco etapas, todas concluídas:

- **Auditoria do código** — leitura completa dos 6 arquivos de código do projeto.
- **Diagnóstico de lentidão e de animações pouco fluidas** — causas medidas no navegador e confirmadas no código.
- **Correções** — desempenho, bugs funcionais, servidor e acessibilidade.
- **Nova paleta** — “dia ensolarado de navegação”, aplicada a CSS, hero, cenas 3D e modal.
- **Pré-visualização e publicação** — capturas em desktop e celular; código no GitHub e **site publicado no GitHub Pages** (HTTPS), com preparação segura para o domínio royalnavy.com.br (seção 11).

Resultados medidos

Indicador	Antes	Depois
FPS medido no painel do navegador (DPR 1,25; WebGL por software)	1,7	56,7 no topo · 60 com os navios visíveis
Contextos WebGL criados ao carregar a página	13, todos renderizando em 60 fps continuamente	0 (criados sob demanda ao chegar perto da tela)
Draw calls do 3D com o hero na tela (nenhum navio visível)	Contínuos (todas as cenas rodavam sempre)	0
Draw calls por frame com o modal aberto	Fundo + modal	≈ 17 (só o modal; fundo pausado)
Redimensionamento de canvas 3D	A cada frame em telas com DPR ≠ 1	Somente quando o tamanho muda
Pesos de fonte baixados do Google Fonts	12	9
Hover dos cards	Não funcionava	Funciona (translateY -8 px)
Barras da seção Frota	Ficavam em 0 para sempre	Animam até o valor correto
Conteúdo do modal (Armamento, História)	Cortado, sem rolagem	Rolável (1192 px de conteúdo em 691 px)
Erros no console do navegador	—	Nenhum, em desktop e celular
Acesso ao site	Só em localhost:5000, na máquina do autor	Público, em HTTPS: alansouzadev7.github.io/Naval_Design-WEB-V.1

Sobre a medição: o painel do navegador do aplicativo renderiza WebGL por software (sem placa de vídeo), então os valores absolutos são pessimistas — o mesmo teste, mesma máquina e mesma configuração, foi usado antes e depois, o que torna a *comparação* válida. Em uma GPU real o ganho tende a ser menor em FPS, mas continua significativo em uso de CPU/GPU e bateria, porque o trabalho contínuo deixou de existir.

Tamanho do código

Arquivo	Antes (linhas)	Depois (linhas)	Observação
static/css/style.css	1569	1579	Reescrito: nova paleta, sem transition: all, reveal sem conflito
static/js/main.js	464	617	Reescrito: hero diurno, scroll único, modal correto
static/js/ships-3d.js	1034	867	Reescrito: gerenciador de cenas + cache + instancing
static/js/ships-data.js	182	182	Sem alterações
templates/index.html	545	554	Edições pontuais (head, modal, atributos)
app.py	41	60	Servidor corrigido
README.md, .gitignore	—	41 + 13	Novos



Figura 1 — Hero com a nova paleta (1366×768): céu, sol, nuvens, gaivotas, veleiro, navio ao fundo e barra de estatísticas.

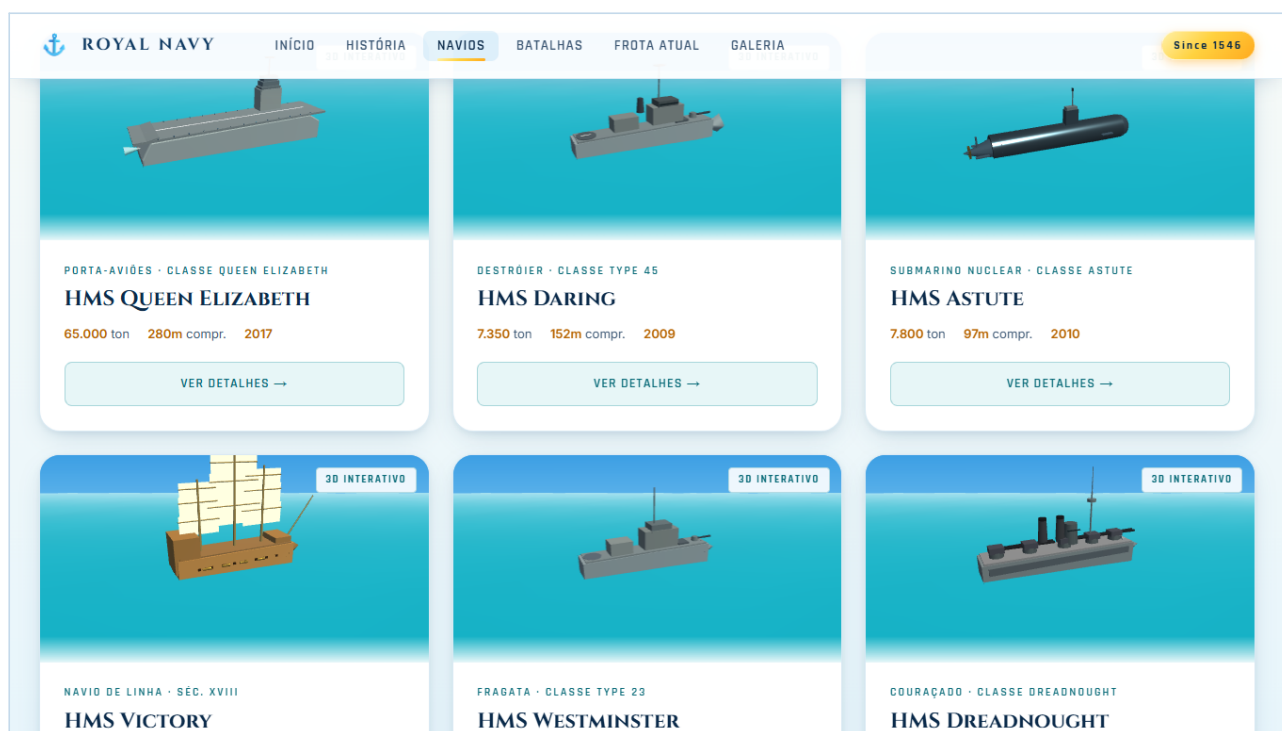
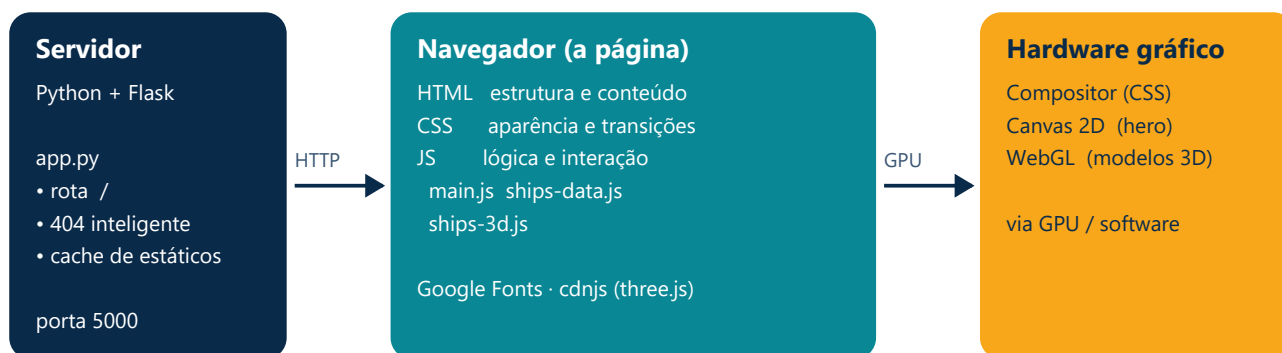


Figura 2 — Seção Navios: cards com cenas 3D de mar ensolarado, iluminadas por sol e céu.

2. Dossiê técnico: como o site foi construído

2.1 Visão geral da arquitetura

O site é uma **página única** (single page) servida por um pequeno servidor Python. O servidor entrega o HTML e os arquivos estáticos; **toda a interatividade acontece no navegador**, em três camadas que trabalham juntas: estrutura (HTML), aparência (CSS) e comportamento (JavaScript), sendo que o JavaScript se divide em quatro responsabilidades distintas.



Fluxo: o navegador pede "/" → Flask devolve index.html → o navegador baixa CSS, fontes e scripts → o JavaScript monta a interação e pede à GPU para desenhar. Depois do primeiro carregamento, o servidor não participa mais.

2.2 Quem faz o quê: o papel de cada tecnologia

Tecnologia	Responsabilidade no projeto	Onde vive
Python 3 + Flask	Servidor web. Entrega a página em /, os arquivos de /static, define cache, trata 404 e decide se o modo debug pode ficar ligado. Não gera conteúdo dinâmico: o template não tem lógica Jinja.	app.py
HTML5	Estrutura semântica e todo o conteúdo em português: navegação, hero, linha do tempo, cards de navios, batalhas, frota, galeria, rodapé e o modal. Define os <canvas> onde o 3D e o hero são desenhados, ganchos de dados (data-*) e atributos de acessibilidade.	templates/index.html
CSS3	Toda a aparência: paleta (variáveis CSS), tipografia, grid/flex, cards, botões, ondas divisórias, responsividade (4 breakpoints), estados de hover/foco, transições e as animações de entrada. Também anima as barras da frota e o modal.	static/css/style.css
JavaScript — interface	Comportamento da página: navbar que muda ao rolar, link ativo, parallax do hero, contadores, revelação dos elementos ao rolar, filtro de navios, barras da frota, modal (abrir/fechar/foco/ESC), brilho do cursor.	static/js/main.js
JavaScript — Canvas 2D	Desenha o hero: céu, sol, nuvens, gaivotas, veleiro, navio ao fundo, ondas e brilhos na água, com animação baseada em tempo.	main.js (initHeroSea)
JavaScript + Three.js (WebGL)	Constrói e renderiza os modelos 3D: 6 tipos de navio, oceano, luz, câmera e interação de arrastar para girar. Gerencia o ciclo de vida das 12 cenas + modal.	static/js/ships-3d.js
JavaScript — dados	Base de dados dos navios (nome, classe, descrição, especificações, armamento, história) que alimenta o modal.	static/js/ships-data.js
SVG (embutido)	Ícone do site (favicon) e a máscara em onda usada nos divisores de seção — ambos como data: URI, sem requisições extras.	index.html, style.css
Serviços externos	Google Fonts (Cinzel, Inter, Rajdhani) e cdnjs (three.js r128, com verificação SRI).	CDN

2.3 Estrutura de arquivos

```

site/
├── app.py                servidor Flask (60 linhas)
├── requirements.txt      flask==3.0.3
├── README.md            uso, publicação e domínio
├── build_static.py       gera docs/ (versão estática)
├── enable_custom_domain.py ativa/remove domínio (trava de DNS)
├── docs/                site estático do GitHub Pages
├── documentacao/        este PDF
├── .gitignore
├── templates/
│   └── index.html       a página inteira (554 linhas)
├── static/
│   ├── css/style.css    estilos e paleta (1579 linhas)
│   └── js/
│       ├── main.js      interface + hero 2D (617 linhas)
│       ├── ships-3d.js  motor 3D Three.js (867 linhas)
│       └── ships-data.js dados dos navios (182 linhas)

```

2.4 Seções da página e o que cada uma usa

Seção (id)	Conteúdo	Tecnologias envolvidas
Navbar	Marca, links âncora, selo "Since 1546"	CSS (fixa, troca de estilo ao rolar) · JS (classe <i>scrolled</i> , link ativo)
Hero (#hero)	Título, chamada, botões, estatísticas	Canvas 2D · CSS (raios de sol, vinheta, animações de entrada) · JS (parallax, contadores)
História (#history)	Linha do tempo com 7 marcos	CSS (grid/flex, cards) · JS (reveal ao rolar, esquerda/direita)
Navios (#ships)	Filtro + 6 cards com 3D	Three.js/WebGL · CSS · JS (filtro, drag, modal)
Batalhas (#battles)	6 batalhas, uma em destaque	CSS (grid, ondas em máscara SVG)
Frota (#fleet)	Cena 3D em formação + 6 categorias	WebGL · CSS (barras animadas por transform)
Galeria (#gallery)	5 cenas 3D (1 grande)	WebGL · CSS grid
Modal	Ficha técnica + 3D em tela cheia	HTML (dialog) · JS (foco, ESC) · WebGL (renderer reutilizado)
Rodapé	Links e créditos	CSS (onda divisória)

2.5 Ordem de carregamento (o que acontece ao abrir a página)

- 1. O navegador pede `/`; o Flask devolve `index.html` ($\approx$ 25 KB).
- 2. Ele lê o `<head>`: ícone embutido, *preconnect* para Google Fonts e cdnjs, folha de fontes (9 pesos) e o CSS ($\approx$ 44 KB). O CSS é bloqueante de renderização — por isso a página já nasce estilizada.
- 3. Os quatro scripts têm *defer*: baixam em paralelo *sem bloquear* a pintura e executam em ordem — `three.js` → `ships-data.js` → `ships-3d.js` → `main.js`.
- 4. `main.js` liga os módulos (`scroll`, `hero`, `contadores`, `reveal`, `filtro`, `modal`...) e chama `ShipEngine.init()`.
- 5. `init()` apenas *registra* os 12 canvases. Nenhuma cena 3D é criada ainda; isso só ocorre quando um canvas chega a 120 px da tela.

2.6 Decisões de projeto

Decisão	Motivo
Manter Flask, HTML/CSS/JS puros e Three.js r128	Preservar a base do projeto; r128 é a última série com o build global <code>three.min.js</code> , que o site usa via CDN.
Paleta em variáveis CSS (<code>:root</code>)	Trocar o tema inteiro editando um bloco; os tokens têm nomes semânticos (<code>sky</code> , <code>sea</code> , <code>sun</code> , <code>navy</code> , <code>coral</code> , <code>sand</code>).
Só <code>transform</code> e <code>opacity</code> em animações	Essas duas propriedades são animadas pelo compositor da GPU, sem recalcular layout nem repintar.
Estado visual por classes , não por <code>style</code> inline	Evita brigas de especificidade (causa de dois bugs originais) e mantém o CSS como fonte única da aparência.
Um único loop de animação 3D	Um ponto de controle para pausar/retomar tudo (aba oculta, modal aberto, fora da tela).
Servidor só em 127.0.0.1 por padrão	O debugger do Werkzeug executa código no servidor; nunca deve ficar exposto na rede.

3. O motor 3D em detalhe

Todo o 3D está em `ships-3d.js`, dentro de uma função anônima que expõe apenas `window.ShipEngine = { init, openModal, closeModal }`. Se o `three.js` não carregar (CDN indisponível), o motor vira um *no-op* e o restante do site continua funcionando.

3.1 Modelos procedurais

Nenhum arquivo de modelo (`.glb`, `.obj`) é usado: cada navio é **montado por código** a partir de primitivas (caixas, cilindros, cones, esferas, planos e uma extrusão para o casco do porta-aviões).

Função	Modelo	Peças notáveis
<code>buildCarrier()</code>	Porta-aviões (Queen Elizabeth)	Casco extrudado, convés, rampa <i>ski-jump</i> , ilha com 3 níveis, radar, 26 luzes de borda
<code>buildDestroyer()</code>	Destróier (Type 45)	Casco + proa, superestrutura, ponte, canhão, VLS, helipad
<code>buildFrigate()</code>	Fragata (Type 23)	Casco, superestrutura, canhão, mastro, helipad
<code>buildSubmarine()</code>	Submarino (Astute)	Casco cilíndrico, cúpula, cone de popa, vela, periscópio, 4 tubos, lemes, hélice
<code>buildGalleon()</code>	Galeão (HMS Victory)	Casco, 3 mastros com 9 velas/vergas, 16 portinholas, 8 canhões, gurupés, bandeira
<code>buildBattleship()</code>	Couraçado (Dreadnought)	Casco, 5 torres de canhões gêmeos, 2 chaminés, mastro com cofa

3.2 Cena, luz e cenário ensolarado

- **Céu:** uma textura de gradiente (2×256 px, criada em um canvas 2D) usada como `scene.background` e **compartilhada** por todas as cenas.
- **Mar:** plano de 80×80 com 30×30 segmentos e ondulação determinística (gerador pseudoaleatório com semente fixa); material opaco turquesa. A linha d'água fica em $y = -0,32$, de modo que os cascos aparecem parcialmente submersos.
- **Luz:** apenas duas — *HemisphereLight* (céu claro/reflexo do mar) e *DirectionalLight* quente (o sol). Antes eram quatro luzes, e cada luz aumenta o custo de cada pixel.
- **Névoa:** exponencial na cor do horizonte, com densidade dividida pelo afastamento da câmera para não "lavar" navios distantes.
- **Sombras:** somente no modal e na imagem grande da galeria (mapa de 1024²). Nos demais quadros, desligadas.

3.3 Ciclo de vida das cenas (o coração da otimização)

Cada `<canvas>` vira uma *entrada* no gerenciador, que decide **quando criar**, **quando desenhar** e **quando parar**:

Evento	Mecanismo	Efeito
Canvas chega a 120 px da tela	IntersectionObserver	A cena entra na fila de criação (no máximo uma por frame , para não travar a rolagem)
Canvas sai da tela	IntersectionObserver	Deixa de ser renderizada (o último quadro permanece visível)
Nenhuma cena visível	Loop verifica <code>active = 0</code>	O <code>requestAnimationFrame</code> é cancelado — zero trabalho
Aba do navegador oculta	<code>visibilitychange</code>	Loop parado; retoma ao voltar

Evento	Mecanismo	Efeito
Modal aberto	Flag <i>backgroundPaused</i>	Só a cena do modal renderiza; o fundo (coberto) pausa
Card esmaecido pelo filtro	Classe <i>is-dimmed</i>	A cena daquele card é pulada
Tamanho do quadro muda	ResizeObserver	setSize e projeção da câmera ajustados uma vez

```
function frame(now) {
  rafId = requestAnimationFrame(frame);
  const dt = Math.min((now - lastTime) / 1000, 0.05); // tempo real, limitado a 50 ms
  if (initQueue.length) initEntry(initQueue.shift()); // 1 cena nova por frame
  let active = 0;
  for (const e of entries) {
    if (!e.ready || !e.visible) continue;
    if (backgroundPaused && !e.isModal) continue;
    if (e.card && e.card.classList.contains('is-dimmed')) continue;
    if (e.resizePending && !applyResize(e)) continue;
    e.update(dt); e.renderer.render(e.scene, e.camera); active++;
  }
  if (!active && !initQueue.length) stop(); // nada a desenhar -> para o loop
}
```

3.4 Reaproveitamento de recursos

- **Cache de geometrias e materiais** (`G.box(...)`, `mat(color, opts)`): um mesmo par forma+materiais é criado *uma vez* e usado por todas as cenas e navios. Isso reduz memória e evita recompilar shaders.
- **InstancedMesh**: peças repetidas (26 luzes do convés, 16 portinholas, 8 canhões, 4 tubos de torpedo, lemes, pás da hélice) viram *um* desenho cada, em vez de dezenas.
- **Renderer do modal criado uma vez** e reutilizado: ao trocar de navio, apenas o grupo 3D é substituído.
- **Canvas opaco** (`alpha:false`): o compositor do navegador não precisa misturar o canvas com o fundo da página.

3.5 Interação

Arrastar para girar usa **Pointer Events** com *pointer capture* (funciona igual para mouse, toque e caneta e não depende de listeners em window). Os canvases têm `touch-action: pan-y`: o gesto vertical continua rolando a página; o horizontal gira o navio. Depois de 2,5 s sem tocar, a rotação automática retorna. Toda a suavização usa $k = 1 - e^{-5 \cdot dt}$, portanto independe da taxa de quadros do monitor.

3.6 Enquadramento automático

Quadros “em pé” (como o painel esquerdo do modal, com proporção $\approx 0,85$) cortavam o navio. A câmera agora se afasta por $k = \sqrt{1,6 / \text{aspecto}}$ sempre que a proporção fica abaixo de 1,6, mantendo o navio inteiro visível.

4. O hero animado e as demais animações

4.1 Hero em Canvas 2D

O fundo do hero é um <canvas> 2D desenhado por `initHeroSea()`. O princípio é: **pré-renderizar tudo que não muda** e desenhar por frame só o que se move.

Elemento	Como é feito	Custo por frame
Céu + mar + névoa do horizonte	Pré-renderizados em um canvas fora da tela (refeito só ao redimensionar)	1 <code>drawImage</code>
Sol	Sprite radial pré-renderizado; leve pulsar por escala	1 <code>drawImage</code>
Nuvens (7)	3 sprites pré-renderizados, deriva horizontal por tempo	7 <code>drawImage</code>
Gaivotas (4)	Duas curvas quadráticas; batida de asa por seno	4 traçados curtos
Navio ao fundo e veleiro	Polígonos simples com balanço leve	Poucos preenchimentos
Ondas (5 camadas)	Senoides somadas, gradientes pré-calculados , brilho na crista	5 preenchimentos
Brilhos do sol na água (34)	Retângulos finos com cintilação individual	34 retângulos

- **Pausa inteligente:** a animação para quando o hero sai da tela (`IntersectionObserver`) ou a aba fica oculta.
- **Tempo real:** o movimento usa dt em segundos — um monitor de 144 Hz não anima mais 2,4× mais rápido que um de 60 Hz (como acontecia com `t += 0.016` por frame).
- **Resolução:** DPR limitado a 1,5 e canvas opaco (`alpha:false`).
- **Quadro estático:** com `prefers-reduced-motion`, o hero é desenhado uma vez, sem loop.

4.2 Demais animações e como cada uma é feita

Animação	Implementação	Por que é fluida
Entrada do hero	CSS <code>@keyframes heroFadeIn</code> com atrasos escalonados	opacity + transform
Parallax do título	Um handler de scroll (rAF) escreve <code>translate3d</code> e opacidade	Sem leitura de layout no scroll; só roda com o hero visível
Navbar ao rolar	Classe <code>scrolled</code> (fundo, sombra e cores)	Transição só de cor/sombra — sem blur
Link ativo	<code>IntersectionObserver</code> com faixa no meio da tela	Nenhum trabalho no scroll
Contadores	rAF + <code>easeOutCubic</code> , 1,8 s, formato pt-BR	Baseado em tempo
Revelação ao rolar	<code>IntersectionObserver</code> adiciona <code>is-visible</code> ; ao fim da transição as classes são removidas	opacity/transform; o elemento volta ao CSS normal e o hover funciona
Hover dos cards	CSS transform + sombra	Compositor
Filtro de navios	Classe <code>is-dimmed</code> + atributo <code>inert</code>	Transição no CSS; itens fora do filtro saem do foco
Barras da frota	<code>scaleX(var(--fill))</code> com atraso escalonado	Compositor (antes animava <code>width</code>)
Modal	Fade do overlay + escala do contêiner; <code>visibility</code> escondido quando fechado	Sem blur; some do fluxo de acessibilidade ao fechar
Brilho do cursor	Um div movido por <code>translate3d</code> ; loop para ao alcançar o cursor	Só com mouse; sem blend-mode

4.3 Acessibilidade incorporada

- `prefers-reduced-motion`: animações e transições praticamente desligadas; hero estático; sem parallax nem brilho.
- Modal: foco vai para o botão fechar, **Tab** fica preso dentro do modal, **Esc** fecha e o foco volta ao elemento que abriu.
- Canvases 3D com `role="img"` e `aria-label`; canvas decorativo do hero com `aria-hidden`.
- Foco visível global (contorno âmbar), botões de filtro com `aria-pressed` e link ativo com `aria-current`.

5. Diagnóstico de desempenho

A causa da lentidão não era um único defeito, e sim a **soma de trabalho contínuo e invisível**: enquanto o usuário olhava o hero, o navegador também renderizava 12 cenas 3D fora da tela, repintava um grid animado, borrava fundos sobre canvases em movimento e recalculava layout a cada evento de rolagem. Cada item abaixo foi identificado por leitura do código; os mais graves foram confirmados por medição no navegador.

5.1 Problemas de maior impacto

#	Problema encontrado	Consequência	Solução aplicada
P1	13 contextos WebGL criados no carregamento, cada um com seu <i>loop</i> a 60 fps, mesmo fora da tela	GPU/CPU ocupadas o tempo todo; pico de travamento no <i>load</i> (13 renderers e compilações de shader); risco de perder contextos (limite $\approx$ 16)	Criação sob demanda + um loop único que só desenha o que está visível
P2	Verificação de <i>resize</i> comparava <i>canvas.width</i> (que já inclui o DPR) com <i>clientWidth</i>	Em telas com escala $\neq$ 100% (ex.: 125%), <i>setSize</i> rodava em todo frame , realocando o buffer de desenho	<i>ResizeObserver</i> : ajusta só quando o tamanho realmente muda
P3	Modal: o loop continuava após fechar; a cada abertura novos listeners em <i>window</i> e um novo renderer eram criados	Trabalho invisível permanente que crescia a cada uso; vazamento de memória	Renderer criado uma vez; loop para ao fechar; listeners por Pointer Events no próprio canvas
P4	12 mapas de sombra (1024 ²) com PCF suave, antialias e <i>tone mapping</i> em todas as cenas	Cada cena pequena pagava por sombras que mal se vêem	Sombras só no modal e na imagem grande; <i>tone mapping</i> removido
P5	Materiais e geometrias novos para cada peça; 26 esferas de luz + 16 portinholas + canhões como malhas separadas	Centenas de draw calls e de objetos por cena	Cache compartilhado + <i>InstancedMesh</i>
P6	<i>backdrop-filter</i> (blur 4–20 px) na navbar, na barra de estatísticas, nos selos 3D e no overlay do modal	O blur é reprocessado a cada mudança do que está atrás — e atrás havia canvases animando	Removido; fundos quase opacos
P7	<i>.hero::before</i> animava <i>background-position</i> em tela cheia, para sempre	Repintura da viewport inteira a cada frame (não é acelerável pelo compositor)	Substituído por raios de sol estáticos (uma única pintura)
P8	Brilho do cursor: loop infinito, alterando <i>left/top</i> e com <i>mix-blend-mode: screen</i> sobre tudo	Layout a cada frame + mistura de cores da página inteira	<i>translate3d</i> , sem <i>blend-mode</i> , loop para ao convergir, só com mouse

5.2 Problemas de impacto médio

#	Problema	Solução
P9	Hero 2D: 6 createLinearGradient por frame, 120 estrelas, partículas, sem pausa fora da tela; tempo por frame ($t += 0.016$)	Gradientes/sprites pré-renderizados; pausa por visibilidade; tempo em segundos (dt)
P10	Handler de scroll lia offsetTop de todas as seções (layout forçado) e escrevia estilos a cada evento	Um handler com rAF que só lê scrollY; link ativo por IntersectionObserver
P11	transition: all em ~20 componentes	Listas explícitas de propriedades (transform, box-shadow, cores...)
P12	Barras da frota animavam width (layout)	transform: scaleX
P13	Contadores com setInterval(16 ms)	rAF com easing por tempo
P14	Animação infinita de escala em uma camada de seção inteira (batalhas), quase invisível	Removida
P15	Scripts sem defer e sem verificação de integridade; 12 pesos de fonte; sem preconnect para o cdnjs	defer + SRI (sha512), 9 pesos, preconnect
P16	Canvases WebGL com alpha:true; oceano transparente; 4 luzes por cena	Canvas opaco; oceano opaco; 2 luzes
P17	Sem prefers-reduced-motion	Suporte completo (CSS e JS)

5.3 Exemplos de código (antes → depois)

P2 — resize a cada frame:

```
// ANTES (todo frame): em DPR 1,25 a condição é SEMPRE verdadeira
if (renderer.domElement.width !== cw || renderer.domElement.height !== ch) {
  renderer.setSize(cw, ch, false); // realoca o buffer de desenho
}
// DEPOIS: ResizeObserver marca "resizePending" só quando o tamanho muda
const ro = new ResizeObserver(list => list.forEach(r => byCanvas.get(r.target).resizePending = true));
```

P7 — grid animado do hero:

```
/* ANTES: repinta a tela inteira a cada frame, para sempre */
.hero::before { background-size: 60px 60px; animation: gridDrift 20s linear infinite; }
@keyframes gridDrift { to { background-position: 60px 60px; } }

/* DEPOIS: raios de sol estáticos, uma única pintura */
.hero::before { background: repeating-conic-gradient(from 195deg at 78% 24%, ...); }
```

6. Erros e bugs corrigidos

Esta seção separa **bugs funcionais** (comportamento errado) dos problemas de desempenho da seção anterior. Os itens marcados com **[R]** foram **reproduzidos no navegador** antes de serem corrigidos; os demais foram identificados por leitura do código e verificados depois da correção.

6.1 Interface e animações

#	Bug	Causa raiz	Correção
B1 [R]	O "levantar" dos cards no hover nunca acontecia (ships, batalhas, linha do tempo, frota)	.fade-in-up.visible e .ship-card:hover têm especificidade (0,2,0) e a primeira vem depois no arquivo, então translateY(0) vencia	Classes de revelação são removidas no fim da transição; o elemento volta ao CSS normal. Hover verificado: matrix(1,0,0,1,0,-8)
B2 [R]	Hover lento (0,7 s) e sem transição de sombra/borda	.fade-in-up definia transition: opacity 0.7s, transform 0.7s e sobrescrevia a do componente	Mesma correção de B1
B3 [R]	Barras da seção Frota ficavam em 0 px para sempre	O JS zerava o style inline <i>antes</i> de ler a largura original; a regex depois só encontrava "0px"	Largura passa a vir de --fill no HTML; o JS só adiciona uma classe; a barra anima por scaleX
B4	Parallax do título não funcionava	A animação CSS com forwards tem precedência sobre estilos inline e travava transform/opacity	Animação removida do contêiner (os filhos animam); parallax via rAF
B5	Erro no console ao clicar nos links do rodapé ("'#' is not a valid selector")	Links href="#" passavam pelo handler de scroll suave, que fazia querySelector('#')	Links usam data-open-ship; scroll suave passou para o CSS
B6	Atributos data-aos sem biblioteca AOS	Código morto (7 elementos)	Removidos
B7	Cards filtrados ainda recebiam foco por teclado	Só se usava pointer-events: none	Atributo inert + aria-pressed nos botões
B8	Em telas baixas o conteúdo do hero passava por trás da barra de estatísticas (botões escondidos)	Barra absoluta sem reserva de espaço	Padding reservado no hero; título limitado por 17vh
B9	Card de destaque (Trafalgar) com título e selo empurrados para o canto	Grid com coluna <i>auto</i> e itens sem posição explícita: o texto longo alargava a coluna	Posições explícitas de coluna/linha; versão de coluna única no celular

6.2 Modal

#	Bug	Causa raiz	Correção
B10	Conteúdo cortado: Armamento e História não apareciam e não rolavam	Linha do grid <i>auto</i> crescia com o conteúdo além do <code>max-height: 90vh</code> ; <code>overflow: hidden</code> cortava	<code>grid-template-rows: minmax(0, 1fr)</code> : o painel direito rola (1192 px em 691 px, medido)
B11	Navio deslocado para baixo e fora do quadro	O canvas seguia a altura inflada da linha do grid; câmera fixa para um quadro largo	Correção do grid + câmera com enquadramento automático
B12	Sem gerenciamento de foco, ESC ou armadilha de Tab; rolagem do fundo saltava ao abrir	Ausentes	Foco no botão fechar, Tab preso, Esc fecha, foco devolvido, <code>scrollbar-gutter: stable</code>
B13	Renderização continuava após fechar; memória crescia a cada abertura	Loop nunca cancelado; <code>render</code> e <code>listeners</code> recriados	Loop pausado ao fechar; <code>render</code> único
B14	Botão fechar com <code>onClick</code> inline e handlers globais	Estilo do código original	Delegação de eventos (<code>data-open-ship</code> , <code>.modal-close</code>)

6.3 Servidor (app.py)

#	Bug	Causa raiz	Correção
B15 [R]	O servidor não iniciava quando a saída não era um console (<code>UnicodeEncodeError</code>)	Banner com caracteres Unicode (traço duplo e emojis de âncora e globo) em cp1252 (Windows)	Banner só ASCII. Verificado: sobe sem <code>-X utf8</code>
B16	Risco de segurança: <code>debug=True</code> com <code>host="0.0.0.0"</code>	O debugger do Werkzeug executa código Python; ficava acessível para toda a rede local	Host padrão 127.0.0.1; debug só é permitido em host local (desligado automaticamente caso contrário)
B17	Rota <code>/static/<path></code> duplicada	O Flask já registra a rota estática automaticamente	Rota removida
B18 [R]	Arquivos inexistentes (ex.: <code>/favicon.ico</code>) recebiam a página inteira (≈ 25 KB) com status 404	O handler 404 sempre renderizava o <code>index</code>	404 real de 9 bytes para <code>/static/*</code> e URLs com extensão; <code>favicon</code> embutido evita a requisição
B19	Sem cache para arquivos estáticos	Configuração padrão	1 h fora do debug; sem cache em debug

6.4 Erros de julgamento meus, corrigidos durante o trabalho

Registro honesto de ajustes feitos depois da primeira versão, encontrados nas capturas de tela:

- Mar 3D “estourado” em ciano neon: a soma das luzes passava de 1,0. Reduzidas as intensidades e escurecida a água.
- Velas do galeão esverdeadas: o hemisfério refletia o mar ciano nas superfícies verticais. Cor de solo neutralizada.
- Ícone das espadas cruzadas invisível nas Batalhas: glifo de texto herdava cor escura sobre fundo azul. Definida cor clara.
- Barra de estatísticas abaixo da dobra em 1366×768: padding do hero reduzido.
- Comentário/commit dizia “12 → 8 fontes”; o correto é 12 → 9 pesos. Corrigido antes do envio.

7. Nova paleta: dia ensolarado no mar

A identidade anterior era noturna (navy profundo, aço e ouro escuro, estrelas). A nova é **diurna**: o que se vê num dia de sol em alto-mar — céu azul, água turquesa, luz dourada, espuma branca e o vermelho-coral das boias. A mudança é global e vive em **variáveis CSS** (:root), o que permite trocar o tema editando um único bloco.

Céu

sky-50 #F3FBFF	sky-100 #E3F4FD	sky-200 #C4E6F9	sky-300 #92D2F3	sky-400 #57B8EA	sky-500 #2A9BDC	sky-600 #1A7DBB
-------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------

Mar turquesa

sea-300 #6FE3D8	sea-400 #2EC9C2	sea-500 #0FA9B1	sea-600 #0A8795	sea-700 #076A78
--------------------	--------------------	--------------------	--------------------	--------------------

Sol

sun-100 #FFF7D9	sun-200 #FFEC9F	sun-300 #FFDB5C	sun-400 #FFC62E	sun-500 #F9A71A	sun-600 #E58A0B	sun-700 #B96500	sun-800 #A35700
--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------

Azul-marinho (texto e rodapé)

navy-900 #0A2A4A	navy-800 #0F3A63	navy-700 #164A7C	navy-600 #1F5D96
---------------------	---------------------	---------------------	---------------------

Coral e areia

coral-400 #FF8060	coral-500 #F2593A	sand-50 #FFFDF6	sand-100 #FDF3DC
----------------------	----------------------	--------------------	---------------------

Onde cada cor é usada

Papel	Cores	Aplicação
Texto principal / títulos	navy-900, navy-800	Títulos de seção, nomes de navios, números
Texto secundário	#3b5673 (7,7:1 sobre branco)	Parágrafos e legendas
Destaques de seção	sea-600 / sea-700	Palavra em destaque nos títulos, classes dos navios, botão “Ver detalhes”
Ação e ênfase	sun-200 ... sun-500	Botão principal, selos, anos da linha do tempo, resultado das batalhas, sublinhado do menu
Rótulos pequenos sobre claro	sun-800, sea-700	Escolhidos para atingir ≥ 4,5:1 em textos pequenos
Interação/alerta	coral-400/500	Ponto da linha do tempo no hover; botão fechar no hover
Superfícies	branco, sky-50/100, sand-50	Cards e fundos de seção, alternando frio e quente
Seção de contraste	gradiente #0e4d86 → #0a6aa0 → #0a8a9c	Batalhas: mar profundo iluminado por um brilho dourado no canto
Rodapé	navy-900	Âncora visual final; separado por uma onda

Elementos de identidade novos

- **Hero diurno:** sol com halo e raios, nuvens à deriva, gaivotas, veleiro, silhueta de um navio de guerra ao horizonte, ondas com crista brilhante e cintilação do sol na água.
- **Cenas 3D ensolaradas:** céu em gradiente, mar turquesa, luz quente do sol, hulls em cinza-naval claro, velas cor de creme.
- **Ondas divisórias:** máscara SVG reutilizável (colorida por background) entre Navios → Batalhas → Frota e antes do rodapé.
- **Vinheta de contraste no hero:** garante leitura do texto branco sobre o céu e o mar.

Acessibilidade de cor

Texto secundário e rótulos pequenos usam tons escuros (sun-800, sea-700, #3b5673) para atingir contraste mínimo de 4,5:1 sobre fundo claro. Sobre o hero (imagem viva), a leitura é apoiada por vinheta escura suave e *text-shadow*. O contraste do texto grande do hero é adequado, mas o parágrafo sobre o mar tem contraste mais modesto; ficou registrado como ponto de atenção na seção 9.



Figura 3 — Batalhas: seção de contraste em azul profundo, card de destaque corrigido e ícone claro.

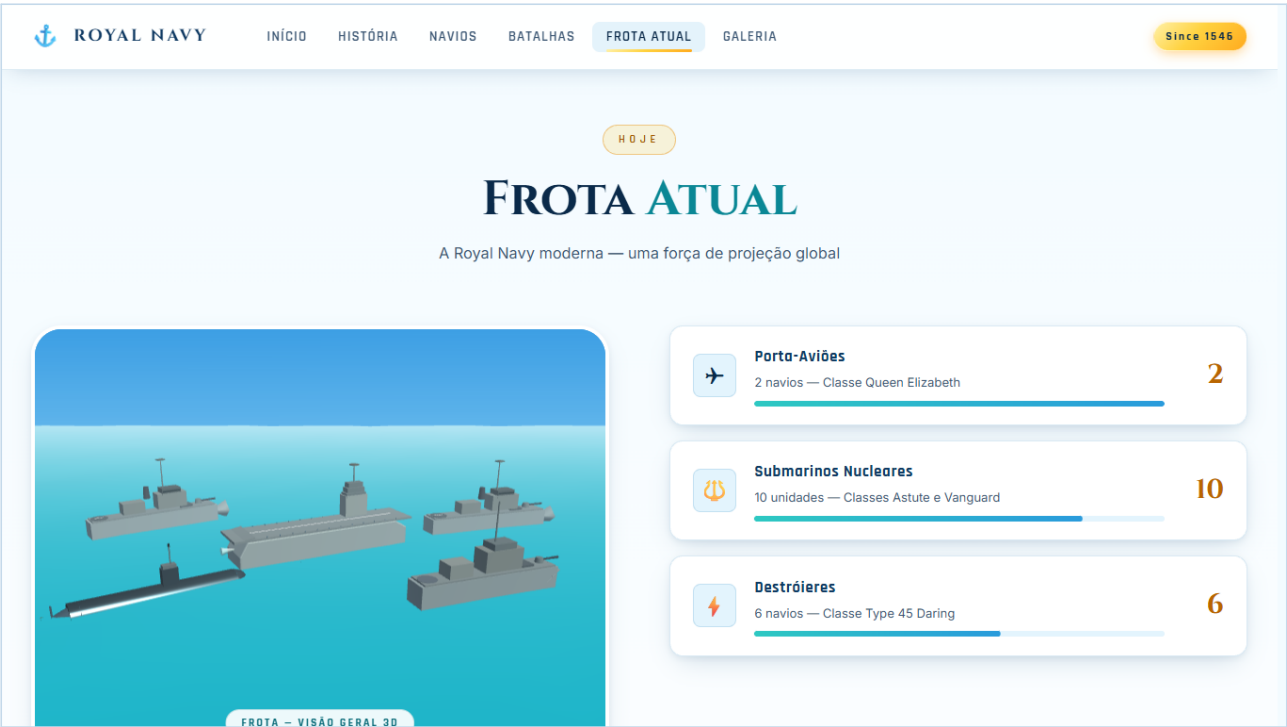


Figura 4 — Frota Atual: cena 3D em formação e barras animadas até seus valores (2, 10 e 6 unidades).

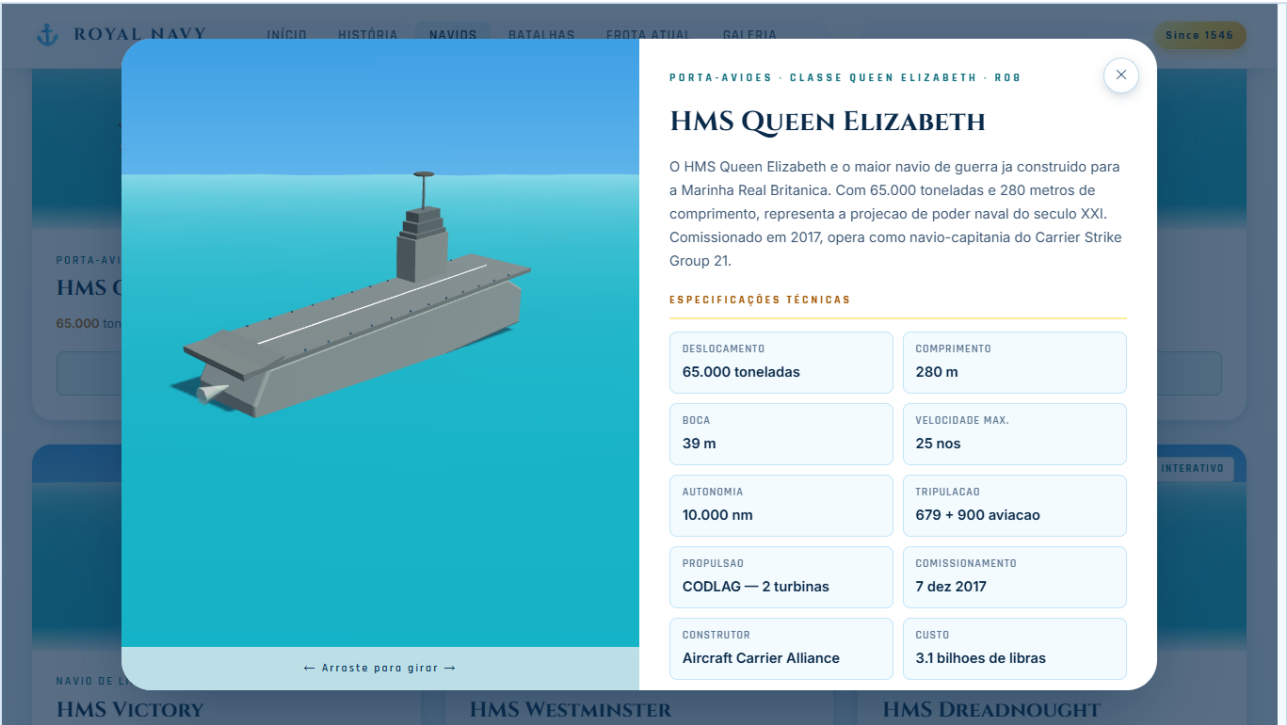


Figura 5 — Modal do HMS Queen Elizabeth: navio enquadrado, ficha técnica e botão fechar.

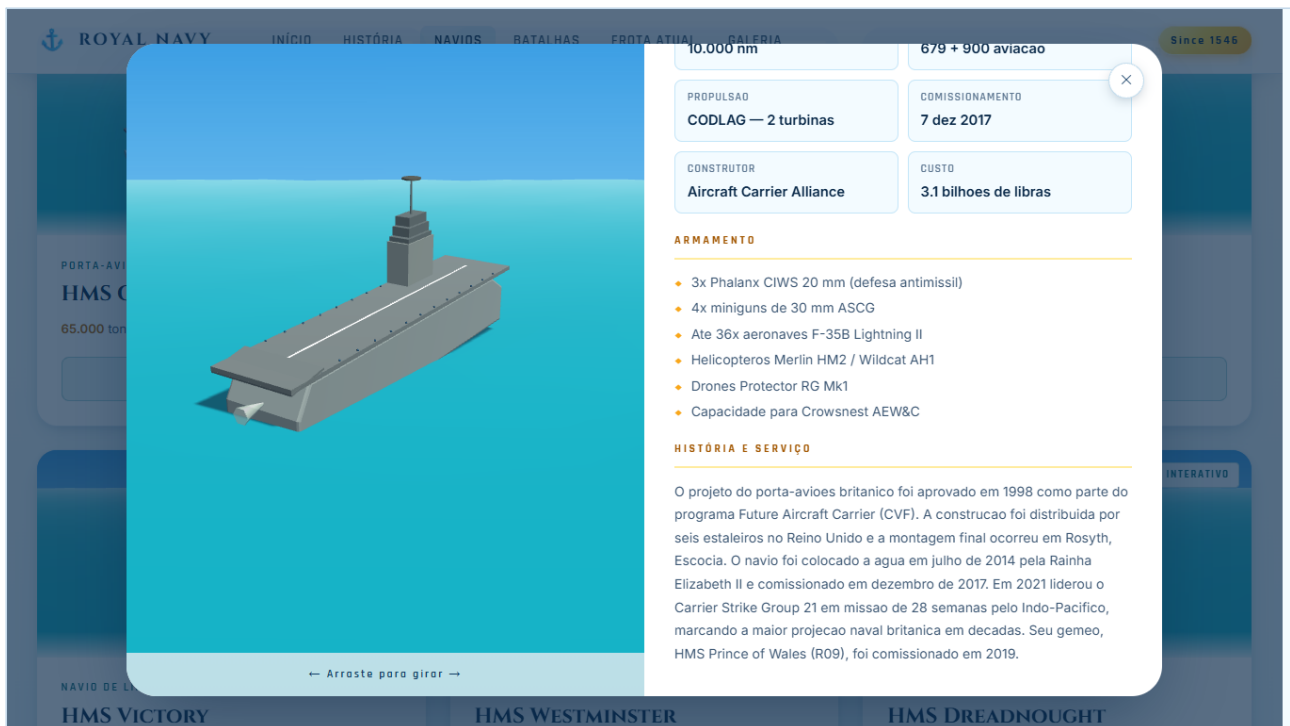


Figura 6 — O painel do modal rolado: Armamento e História agora acessíveis.

8. Como foi verificado

8.1 Métodos

Método	Ferramenta	O que confirmou
Leitura completa do código	Revisão manual (6 arquivos)	Lista de problemas P1–P17 e B1–B19
Medição de FPS	Navegador do aplicativo (rAF durante 3 s)	1,7 FPS (original) → 56,7 (topo) / 60 (navios visíveis)
Inspeção do CSS calculado	<code>document.styleSheets + getComputedStyle</code>	Conflito <code>.fade-in-up × :hover</code> (B1/B2); barras em 0 px (B3)
Contagem de draw calls	Interceptação de <code>drawElements/drawArrays</code> em WebGL/WebGL2	≈ 77/frame com navios; ≈ 17/frame só o modal; 0 no topo
Capturas por seção	Edge headless via Chrome DevTools Protocol (script próprio)	Revisão visual de todas as seções em 1366×768 e 390×844
Hover real	Evento de mouse sintético do DevTools	Transform do card = <code>translateY(-8 px)</code>
Rolagem do modal	Medição de <code>scrollHeight × clientHeight</code> (desktop)	1192 px de conteúdo em 691 px visíveis; <code>overflow-y: auto</code>
Console	Captura de erros e avisos em cada execução	Nenhum erro ou aviso
Rotas do servidor	<code>curl</code>	<code>/</code> = 200; <code>/static/*</code> = 200; arquivo inexistente = 404 (9 bytes); URL desconhecida = 404 + página
Sintaxe	<code>node --check, py_compile</code>	Todos os arquivos válidos
Versão estática num subcaminho	<code>http.server</code> + Edge headless	Site idêntico ao do Flask, sem erros
Política CSP com teste de controle	Domínio <i>Log</i> do DevTools	Política real: 0 violações. Política propositalmente errada: bloqueio detectado (o teste enxerga violações)
Site publicado	<code>curl</code> no endereço do Pages	<code>index.html</code> e os arquivos de static respondendo HTTP 200 (20/09/2026)
Disponibilidade do domínio	RDAP do Registro.br + <code>nslookup</code>	<code>royalnavy.com.br</code> não registrado (404 no RDAP; controle <code>google.com.br</code> = 200; DNS inexistente)
Trava do domínio	Execução de <code>enable_custom_domain.py</code>	Recusou domínio inexistente e domínio de outro servidor; nada foi gravado

8.2 Limitações honestas

- As medições de FPS foram feitas em **WebGL por software**; não foram feitas na sua máquina com GPU real.
- Não houve teste em navegadores diferentes do Chromium/Edge (Firefox e Safari não foram executados).
- Não há testes automatizados no projeto; a verificação foi por medição, captura e inspeção.
- O celular foi verificado por emulação (390×844), não em dispositivo físico; toque e gestos não foram exercitados em hardware real.

- O código original não estava em um repositório git, então não há histórico anterior; as versões antigas podem ser recuperadas pelo histórico do OneDrive.

9. Pendências e recomendações

Prioridade	Item	Detalhe
Alta	Acentuação em ships-data.js	O arquivo de dados está sem acentos ("Porta-Avioes", "Britanica", "projecao"), diferente do restante do site. Não foi alterado por estar fora do escopo pedido; é uma correção simples e de alto valor.
Alta	Menu para celular	Abaixo de 768 px os links do menu somem e não há alternativa (hambúrguer). A navegação por âncoras fica indisponível.
Alta	Registrar royalnavy.com.br	Concluir a compra no Registro.br (só o titular pode) e criar os registros DNS; depois rodar enable_custom_domain.py. Passo a passo na seção 11. O domínio estava disponível em 20/09/2026.
Concluída	Publicar como site	Feito: versão estática em docs/ servida pelo GitHub Pages em HTTPS (seção 11).
Média	Contraste do parágrafo do hero	O texto branco médio sobre o mar tem contraste modesto; considerar aumentar a vinheta ou o peso da fonte.
Média	Hospedar three.js localmente	Hoje vem do cdnjs (com SRI). Hospedar em static/ elimina uma dependência externa e funciona offline (≈ 600 KB).
Média	Compressão gzip/brotli	O servidor de desenvolvimento não comprime; em produção use um servidor/host que comprima (o GitHub Pages já faz).
Baixa	Limite de contextos WebGL	O site usa até 13 contextos após rolar tudo (limite ≈ 16). Uma evolução seria um único renderer compartilhado, ou liberar cenas muito distantes.
Baixa	Testes automatizados	Um pequeno teste de fumaça (Playwright) para carregar a página, checar o console e abrir o modal evitaria regressões.
Baixa	Quebra de linha do texto do hero	O fixo gera linhas curtas no celular.

10. Operação: executar, versionar e publicar

10.1 Executar localmente

```
python -m venv .venv
.venv\Scripts\activate           # Windows (Linux/macOS: source .venv/bin/activate)
pip install -r requirements.txt
python app.py                    # abre em http://localhost:5000
```

10.2 Variáveis de ambiente

Variável	Padrão	Função
PORT	5000	Porta do servidor
HOST	127.0.0.1	Interface de rede. 0.0.0.0 expõe na rede local; nesse caso o modo debug é desligado automaticamente
FLASK_DEBUG	1	0 desliga o recarregamento automático e ativa cache de 1 h nos estáticos

10.3 Repositório

Item	Valor
URL	https://github.com/AlanSouzaDev7/Naval_Design-WEB-V.1
Branch	main (rastreia origin/main)
Site publicado	https://alansouzadev7.github.io/Naval_Design-WEB-V.1/ (Pages: branch main, pasta /docs)
Commits	a450870 — otimização e paleta · f26ebc1 — versão estática e segurança · mais o commit desta documentação e do preparo do domínio (ver <code>git log</code>)
Arquivos versionados	código, README, .gitignore, requirements, scripts de build/domínio, docs/ e este PDF
Fora do repositório	.venv, .claude/, arquivo teste, caches Python

10.4 Guia rápido de manutenção

Quero...	Edite...
Trocar as cores do site	Bloco <code>:root</code> no início de <code>style.css</code> (e as cores de <code>C</code> em <code>ships-3d.js</code> para as cenas 3D; e os gradientes do hero em <code>initHeroSea</code>)
Adicionar um navio	1) Entrada em <code>ships-data.js</code> ; 2) card em <code>index.html</code> com <code>data-ship</code> e <code>data-type</code> ; 3) se for um tipo novo, uma função <code>buildX()</code> e um case em <code>buildShip</code>
Ajustar o brilho/luz 3D	<code>createScene()</code> em <code>ships-3d.js</code> (<code>HemisphereLight</code> e <code>DirectionalLight</code>)
Mudar textos e seções	<code>templates/index.html</code>
Mudar cache/porta/segurança	<code>app.py</code> (Flask local) e a meta CSP em <code>index.html</code> (vale também no Pages)
Atualizar o site publicado	Edite <code>templates/static</code> , rode <code>python build_static.py</code> , faça commit e push (o Pages atualiza sozinho em ~1 min)
Ativar o domínio próprio	<code>python enable_custom_domain.py royalnavy.com.br</code> — só depois do registro e do DNS
Adicionar animação de entrada	Inclua o seletor na lista <code>groups</code> de <code>initReveal()</code> (<code>main.js</code>)

11. Publicação, domínio e segurança

11.1 Por que o localhost:5000 não funcionava no GitHub

`localhost` significa “este computador”. O endereço `http://localhost:5000` só existe na máquina de quem executa `python app.py` e apenas enquanto o servidor estiver aberto. Quem abre esse link em outro computador (ou no celular) tenta falar com a *própria* máquina, onde não há nada rodando.

Afirmação	Realidade
“O GitHub roda o meu site”	O GitHub armazena o código. Ele não executa Python nem o Flask.
“localhost é um endereço público”	É um endereço privado da máquina local (127.0.0.1); ninguém de fora consegue acessá-lo.
“Preciso de um servidor Flask ligado”	Não. O template não usa lógica Jinja e nenhuma rota devolve dados dinâmicos: o site é 100% estático.

Solução adotada: publicar o site como **estático** no GitHub Pages — gratuito para repositórios públicos, com HTTPS automático e sem nenhum servidor seu ligado.

11.2 Como a publicação funciona

```

templates/index.html + static/      (fonte de verdade, usada também pelo Flask local)
    |
    | python build_static.py        troca "../static/" por "static/" e copia os arquivos
    v
docs/                               versão estática pronta (index.html, static/, .nojekyll)
    |
    | git push -> GitHub Pages (Settings > Pages: branch main, pasta /docs)
    v
https://alansouzadev7.github.io/Naval_Design-WEB-V.1/    (HTTPS, atualiza em ~1 min)

```

- **Por que reescrever os caminhos:** no Flask a página fica em / e ../static/ resolve para /static/. No Pages o site vive em um subcaminho (/Naval_Design-WEB-V.1/), então caminhos relativos simples (static/...) são necessários.
- **.nojekyll:** desliga o processamento Jekyll do Pages, que ignoraria arquivos e pastas iniciados por "_".
- **Uma só fonte:** a pasta docs/ é *gerada*; edite sempre templates/ e static/ e rode o build.
- **Verificação:** a versão estática foi testada num subcaminho local antes do envio e, depois de ativada, o endereço público respondeu HTTP 200 para a página e para CSS e JS.

11.3 Segurança

O maior ganho de segurança vem da própria arquitetura: um site estático **não tem servidor, banco de dados, login, formulários nem credenciais** para atacar. Sobre isso foram aplicadas camadas adicionais:

Medida	Onde	O que protege
HTTPS (TLS) automático	GitHub Pages	Criptografa o tráfego e impede adulteração no caminho
Content-Security-Policy (CSP)	<meta> em index.html (vale no Pages e no Flask)	Limita de onde scripts, estilos, fontes e imagens podem vir; reduz o impacto de injeção de código (XSS)
SRI (sha512) no three.js	Tag <script>	O navegador recusa o script se o arquivo do CDN for alterado
Referrer-Policy	meta + cabeçalho Flask	Envia o mínimo de informação de origem a terceiros
X-Content-Type-Options, X-Frame-Options, Permissions-Policy	Cabeçalhos do Flask (uso local)	Sem "adivinhação" de tipos, sem iframes e sem câmera/microfone/localização
Debug só em 127.0.0.1	app.py	O debugger do Werkzeug executa código; nunca fica exposto na rede
Repositório sem segredos	Todo o código	Não há chaves, senhas ou tokens no código (o push usou o login do próprio Git)

Política CSP aplicada, diretiva por diretiva

Diretiva	Valor	Motivo
default-src	'self'	Padrão restritivo: só o próprio site
script-src	'self' https://cdnjs.cloudflare.com	Scripts do site e o three.js do cdnjs; nenhum script inline
style-src	'self' 'unsafe-inline' https://fonts.googleapis.com	CSS do site e do Google Fonts; <i>unsafe-inline</i> é necessário para os atributos <i>style</i> (ex.: <i>--fill</i> das barras)
font-src	https://fonts.gstatic.com	Arquivos de fonte do Google
img-src	'self' data:	Ícone e máscara de onda embutidos como <i>data:</i> URI
connect-src	'self'	Sem chamadas de rede a terceiros
object-src / base-uri / form-action	'none' / 'self' / 'self'	Bloqueia plugins e evita sequestro de links e formulários

Teste da política: com a CSP real, o navegador não registrou nenhuma violação. Como prova de que o teste enxerga violações, uma cópia com a política propositalmente restritiva foi bloqueada de fato — e o site degradou sem quebrar (o 3D foi desativado com um aviso).

Riscos residuais (transparência)

- **Sem cabeçalhos HTTP no Pages:** o GitHub Pages não permite definir cabeçalhos; por isso a CSP vai em *meta*, e a diretiva *frame-ancestors* (anti-clickjacking) não pode ser usada nesse modo.
- **Dependência de terceiros:** cdnjs e Google Fonts. O SRI protege o script; as fontes revelam o IP do visitante ao Google (hospedar as fontes localmente eliminaria isso).
- **'unsafe-inline' em estilos:** aceito por ser só CSS de apresentação; não permite executar scripts.

11.4 Domínio personalizado: royalnavy.com.br

Resultado da investigação (20/09/2026): o domínio **não está registrado**. Isso foi verificado por vias independentes:

Verificação	Resultado	Interpretação
RDAP do Registro.br (royalnavy.com.br)	HTTP 404	Sem registro no Registro.br
Controle: RDAP de google.com.br	HTTP 200	A consulta funciona e diferencia registrado de livre
DNS (nslookup royalnavy.com.br e www)	Non-existent domain	Não há zona DNS
Variações (royalnavy.br, royal-navy.com.br, royalnavybr.com.br, marinhareal.com.br)	HTTP 404	Também disponíveis na data da consulta

O que não foi feito, e por quê. Registrar um domínio é uma compra em nome de uma pessoa (CPF/CNPJ) com pagamento; só o titular pode fazê-la. Além disso, **o arquivo CNAME não foi publicado:** com o domínio inexistente, o endereço `github.io` passaria a redirecionar para um endereço que não abre (o site cairia) e, enquanto ninguém o registrasse, qualquer pessoa poderia tomá-lo e receber os visitantes do site.

Trava de segurança criada

O script `enable_custom_domain.py` só grava o CNAME depois de conferir, por DNS, que o domínio já aponta para os quatro IPs do GitHub Pages. Foi testado: recusou `royalnavy.com.br` (não resolve) e `example.com` (resolve para outro servidor), sem gravar nada.

```
python enable_custom_domain.py royalnavy.com.br # ativa (recusa se o DNS ainda não estiver certo)
python enable_custom_domain.py --remove         # volta ao endereço github.io
```

Passo a passo para ativar

- **1. Registrar** royalnavy.com.br em registro.br (conta, CPF/CNPJ, pagamento da anuidade; confirme o valor vigente no site).
- **2. Criar os registros DNS** (tabela abaixo) na zona do domínio.
- **3. Aguardar a propagação** (minutos a algumas horas) e conferir com `nslookup royalnavy.com.br`.
- **4. Ativar no projeto:** rodar o script, depois `git add -A`, `git commit` e `git push`.
- **5. No GitHub** (Settings → Pages): conferir o domínio e marcar **Enforce HTTPS** quando o certificado for emitido (até ~1 h).
- **6. Verificar o domínio** na conta (Settings → Pages → Add a domain, registro TXT), impedindo que terceiros o usem em outro repositório.

Tipo	Nome	Valor	Obrigatório
A	@	185.199.108.153 · 185.199.109.153 · 185.199.110.153 · 185.199.111.153 (um registro por IP)	Sim
AAAA	@	2606:50c0:8000::153 · 2606:50c0:8001::153 · 2606:50c0:8002::153 · 2606:50c0:8003::153	Opcional (IPv6)
CNAME	www	alansouzadev7.github.io	Sim (para o www funcionar)

IPs conferidos na API oficial do GitHub (api.github.com/meta, campo *pages*) em 20/09/2026.

Solução de problemas

Sintoma	Causa provável	O que fazer
O script responde "BLOQUEADO ... não resolve"	Domínio não registrado ou DNS ainda não propagou	Conferir o registro e os registros A; aguardar e repetir
O script responde que resolve para outros IPs	Registros A apontando para outro servidor (ou proxy)	Trocar pelos 4 IPs do GitHub; desligar proxies/CDN que ocultem o IP
Pages mostra "DNS check unsuccessful"	Propagação incompleta ou www sem CNAME	Esperar e usar "Check again"; revisar o CNAME do www
"Enforce HTTPS" desativado	Certificado ainda em emissão	Aguardar até ~1 h; se persistir, remover e recolocar o domínio
O site abre sem estilo/3D depois de ativar o domínio	Cache ou caminhos	Ctrl+F5; os caminhos são relativos e não mudam com o domínio
Quero voltar ao endereço github.io	—	<code>enable_custom_domain.py --remove</code> , <code>commit</code> , <code>push</code> e limpar o domínio em Settings → Pages

11.5 Checklist de publicação

Item	Estado (20/09/2026)
Código no GitHub (repositório público)	Concluído
Versão estática em docs/ + .nojekyll	Concluído
GitHub Pages ativado (main /docs) e respondendo HTTP 200	Concluído (ativado pelo titular; verificado por curl)
HTTPS	Concluído (automático no endereço github.io)
CSP, SRI e cabeçalhos	Concluído e testado
Registro de royalnavy.com.br	Pendente — depende do titular (compra)
Registros DNS (A, AAAA, CNAME www)	Pendente — após o registro
enable_custom_domain.py + push	Pendente — após o DNS propagar
Enforce HTTPS e verificação do domínio	Pendente — após ativar o domínio

12. Glossário

Termo	Significado
Canvas 2D	Elemento HTML em que o JavaScript desenha pixels diretamente (usado no hero).
WebGL	API do navegador para desenhar gráficos 3D na GPU. O Three.js é uma biblioteca que a simplifica.
Three.js	Biblioteca JavaScript de 3D: cena, câmera, luzes, geometrias e materiais.
Draw call	Uma ordem de desenho enviada à GPU. Muitas por frame = mais custo de CPU.
InstancedMesh	Objeto Three.js que desenha várias cópias da mesma peça em uma única draw call.
DPR (device pixel ratio)	Razão entre pixels físicos e CSS. Em escala de 125%, DPR = 1,25.
FPS	Quadros por segundo; 60 é o alvo de fluidez em telas comuns.
requestAnimationFrame (rAF)	Agenda um desenho sincronizado com a taxa de atualização da tela.
dt (delta time)	Tempo real entre dois frames; usado para o movimento não depender do monitor.
IntersectionObserver	API que avisa quando um elemento entra ou sai da tela, sem ler layout a cada scroll.
ResizeObserver	API que avisa quando um elemento muda de tamanho.
Compositor	Etapas do navegador que combina camadas na GPU. transform e opacity animam nele sem repintar.
Repaint / Layout	Repintar pixels / recalcular posições. Ambos são caros quando ocorrem em todo frame.
Especificidade CSS	Regra que decide qual declaração vence quando várias se aplicam ao mesmo elemento.
backdrop-filter	Efeito de desfoque do que está atrás de um elemento; caro sobre conteúdo animado.
SRI	Subresource Integrity: hash que o navegador confere para garantir que um script externo não foi alterado.
defer	Atributo do script que o baixa em paralelo e executa após o HTML, sem bloquear a página.
Debugger do Werkzeug	Console interativo do Flask em modo debug; permite executar código no servidor.
prefers-reduced-motion	Preferência do sistema por menos animação; o site a respeita.
inert	Atributo HTML que remove um elemento do foco, cliques e leitores de tela.
GitHub Pages	Hospedagem gratuita de sites estáticos a partir de um repositório do GitHub.
Site estático	Conjunto de arquivos (HTML, CSS, JS) entregues como estão, sem processamento no servidor.
localhost / 127.0.0.1	Endereço da própria máquina. Não é acessível por outras pessoas.
CSP	Content-Security-Policy: regra que diz ao navegador de quais origens a página pode carregar recursos.
HTTPS / TLS	Conexão criptografada entre navegador e servidor; o Pages a fornece automaticamente.
DNS	Sistema que traduz nomes (royalnavy.com.br) em endereços IP.
Registro A / AAAA	Registros DNS que ligam um nome a endereços IPv4 / IPv6.
Registro CNAME	Registro DNS que faz um nome (www) apontar para outro nome (alansouzadev7.github.io).
Propagação de DNS	Tempo para as mudanças de DNS chegarem a todos os servidores (minutos a horas).

Termo	Significado
RDAP	Protocolo de consulta a dados de registro de domínios; usado para checar a disponibilidade.
NXDOMAIN	Resposta DNS "domínio inexistente".
Enforce HTTPS	Opção do Pages que redireciona todo acesso HTTP para HTTPS.

— Fim do documento —